

VLLM专题（二十六）—使用 Docker

原创

无声之钟

已于 2025-03-19 10:26:34 修改

阅读量693

收藏

点赞数

版权

分类专栏：

大模型专题系列

 文章标签：

docker

容器

人工智能

 大模型专题系列 专栏收录该内容

您已是超级会员，正在免费阅读会员专享内容

查看更多超级会员权益

1. 使用 vLLM 的官方 Docker 镜像

vLLM 提供了一个官方 Docker 镜像用于部署。该镜像可用于运行与 OpenAI 兼容的服务器，并可在 Docker Hub 上获取，名称为 `vllm/vllm-openai`。

```
docker run --runtime nvidia --gpus all \
-v ~/.cache/huggingface:/root/.cache/huggingface \
--env "HUGGING_FACE_HUB_TOKEN=<secret>" \
-p 8000:8000 \
--ipc=host \
vllm/vllm-openai:latest \
--model mistralai/Mistral-7B-v0.1
```

您可以在镜像标签（`vllm/vllm-openai:latest`）之后添加任何其他所需的引擎参数（Engine Arguments）。

注意

您可以使用 `ipc=host` 标志或 `--shm-size` 标志来允许容器访问主机的共享内存。vLLM 使用 PyTorch，而 PyTorch 在底层使用共享内存来在进程之间共享数据，特别是在张量并行推理中。

注意

为了避免许可问题（例如 Issue #8030），可选依赖项未包含在镜像中。如果您需要使用这些依赖项（并已接受许可条款），可以在基础镜像之上创建一个自定义 Dockerfile，并通过额外的层安装这些依赖项：

¥49.90

¥99.00

98 篇文章

已订阅

8折续

```
FROM vllm/vllm-openai:v0.7.3

# e.g. install the `audio` and `video` optional dependencies
# NOTE: Make sure the version of vLLM matches the base image!
RUN uv pip install --system vllm[audio,video]==0.7.3
```

提示

某些新模型可能仅在 Hugging Face Transformers 的主分支上可用。
要使用 Transformers 的开发版本，可以在基础镜像之上创建一个自定义 Dockerfile，并通过额外的层从源代码安装其代码：

```
FROM vllm/vllm-openai:latest

RUN uv pip install --system git+https://github.com/huggingface/transformers.git
```

2. 从源代码构建 vLLM 的 Docker 镜像

您可以通过提供的 Dockerfile 从源代码构建并运行 vLLM。构建 vLLM 的步骤如下：

1. 可选参数：指定 `--build-arg max_jobs=8` 和 `--build-arg nvcc_threads=2`。
2. 使用以下命令构建镜像：

```
DOCKER_BUILDKIT=1 docker build . --target vllm-openai --tag vllm/vllm-openai
```

注意

默认情况下，vLLM 会为所有 GPU 类型构建，以实现最广泛的兼容性。如果您仅为当前机器运行的 GPU 类型构建，可以添加参数 `--build-arg torch_cuda_arch_list=""`，vLLM 将自动检测当前 GPU 类型并为其构建。

如果您使用的是 Podman 而不是 Docker，则可能需要在运行 `podman build` 命令时添加 `--security-opt label=disable` 来禁用 SELinux 标签，以避免某些已知问题。

3. 为 Arm64/aarch64 构建

可以为 Nvidia Grace-Hopper 等 aarch64 系统构建 Docker 容器。截至撰写本文时，这需要使用 PyTorch Nightly 版本，并应被视为实验性功能。使用 `--platform "linux/arm64"` 标志将尝试为 arm64 构建。

注意

由于需要编译多个模块，此过程可能需要一些时间。建议使用 `--build-arg max_jobs=` 和 `--build-arg nvcc_threads=` 标志来加速构建过程。但请确保 `max_jobs` 显著大于 `nvcc_threads` 以获得最佳效果。同时，注意并行作业的内存使用情况，因为它可能会占用大量内存（见以下示例）。

```
Example of building on Nvidia GH200 server. (Memory usage: ~15GB, Build time: ~1475s / ~25 min, Image size: 6.93GB)
python3 use_existing_torch.py
DOCKER_BUILDKIT=1 docker build . \
  --target vllm-openai \
  --platform "linux/arm64" \
  -t vllm/vllm-gh200-openai:latest \
  --build-arg max_jobs=66 \
  --build-arg nvcc_threads=2 \
  --build-arg torch_cuda_arch_list="9.0+PTX" \
  --build-arg vllm_fa_cmake_gpu_arches="90-real"
```

4. 使用自定义构建的 vLLM Docker 镜像

要使用自定义构建的 Docker 镜像运行 vLLM，请执行以下命令：

```
docker run --runtime nvidia --gpus all \
  -v ~/.cache/huggingface:/root/.cache/huggingface \
  -p 8000:8000 \
  --env "HUGGING_FACE_HUB_TOKEN=<secret>" \
  vllm/vllm-openai <args...>
```

参数 `vllm/vllm-openai` 指定了要运行的镜像，应替换为自定义构建的镜像名称（即构建命令中的 `-t` 标签）。

注意

仅适用于版本 0.4.1 和 0.4.2 - 这些版本的 vLLM Docker 镜像需要以 root 用户运行，因为在运行时需要加载 root 用户主目录下的库，即 `/root/.config/vllm/nccl/cu12/libnccl.so.2.18.1`。如果您以其他用户身份运行容器，可能需要先更改该库（及其所有父目录）的权限，以允许该用户访问，然后使用环境变量

`VLLM_NCCL_SO_PATH=/root/.config/vllm/nccl/cu12/libnccl.so.2.18.1` 运行 vLLM。



无声之钟

关注



专栏目录

已订阅

使用 vllm docker 容器部署大语言模型

yuanlulu 的博客 5942

假设 `deepseek-ai/DeepSeek-R1-Distill-Qwen-32B` 被下载到了 `/mnt/llm_deploy/` 目录下，则模型的绝对路径是 `/mnt/llm_deploy/DeepSeek-R1-Distill-Qwen-32B`，后面部署会用到这个目录。上面映射了目录 `/mnt/llm_deploy/` 到 容器内 `./` 部的 `/home/llm_deploy`，则 容器内看到的模型目录是 `/home/llm_deploy/DeepSeek-R1-Distill-Qwen-32B`。搜索最新的 `vllm` 镜像。



docker+ vllm 快速上手 以部署Qwen2.5通义千问为例

最新发布

kesanzz的博客



1万+

一文教你快速上手 vllm + docker, 并部署最强开源模型通义千问



通过 vllm框架进行大模型推理

weixin_45920955的博客



4995

通过 vLLM项目中的 `dockerfile` 文件构建 `docker` 镜像并运行 容器从 `modelscope` 上拉取模型加载并启动 `openai_api` 接口服务